

Client Server

[Start Assignment](#)

- Due No Due Date
- Points 40
- Submitting a file upload

Assignment Overview

Overview

The goal of this assignment is to write a simple **client/server application** that does recursive digit sums. A digit sum of a natural number is the sum of all of its digits. For example, the digit sum of 51289 is $5 + 1 + 2 + 8 + 9 = 25$. If we are doing a recursive digit sum, then the next step would be to do the digit sum of 25: $2 + 5 = 7$.

This assignment will consist of two separate applications: the **server**, and the **client**.

Server Application

The server will:

- Receive input in the form of a string, consisting solely of digits.
 - If the received string contains any non-digit character, the server will return an error code to the client and terminate.
- Perform a digit sum on the number.
- Send the digit sum result to the client.
- Terminate if the input is a single digit long.

Client Application

The client will:

- Request the Server's IP Address and Port.
- Obtain the input string to send to the server from the user.
- If the server returns an error code or a single digit number, exit the client application.
- Send the returned results back to the server, and repeat until the server returns a single digit number.

Submission Requirements:

- Source code for both the server and client applications that is **heavily commented**.
- If non-standard libraries/packages/modules are used, please include a comment explaining which so that someone else can successfully run your code.

Sample Server Output 1

Digit Sum Server Started

Listening on TCP 4242

Received string "Moose"

Invalid Input, Exiting Server Application

Sample Server Output 2

Digit Sum Server Started

Listening on TCP 4242

Received string "91823449"

Sending Digit Sum result: 40

Received string "40"

Sending Digit Sum result: 4

Digit sum is a single digit, Exiting Server Application

Sample Client Output 1

Digit Sum Client Started

Server IP >> 127.0.0.1

Server Port >> 4242

Digit Sum starting string >> Moose

Server returned "Not A Number error, Exiting Client Application

Sample Client Output 2

Digit Sum Client Started

Server IP >> 127.0.0.1

Server Port >> 4242

Digit Sum starting string >> 91823449

Received Digit Sum result: 40

Sending 40 to Digit Sum Server

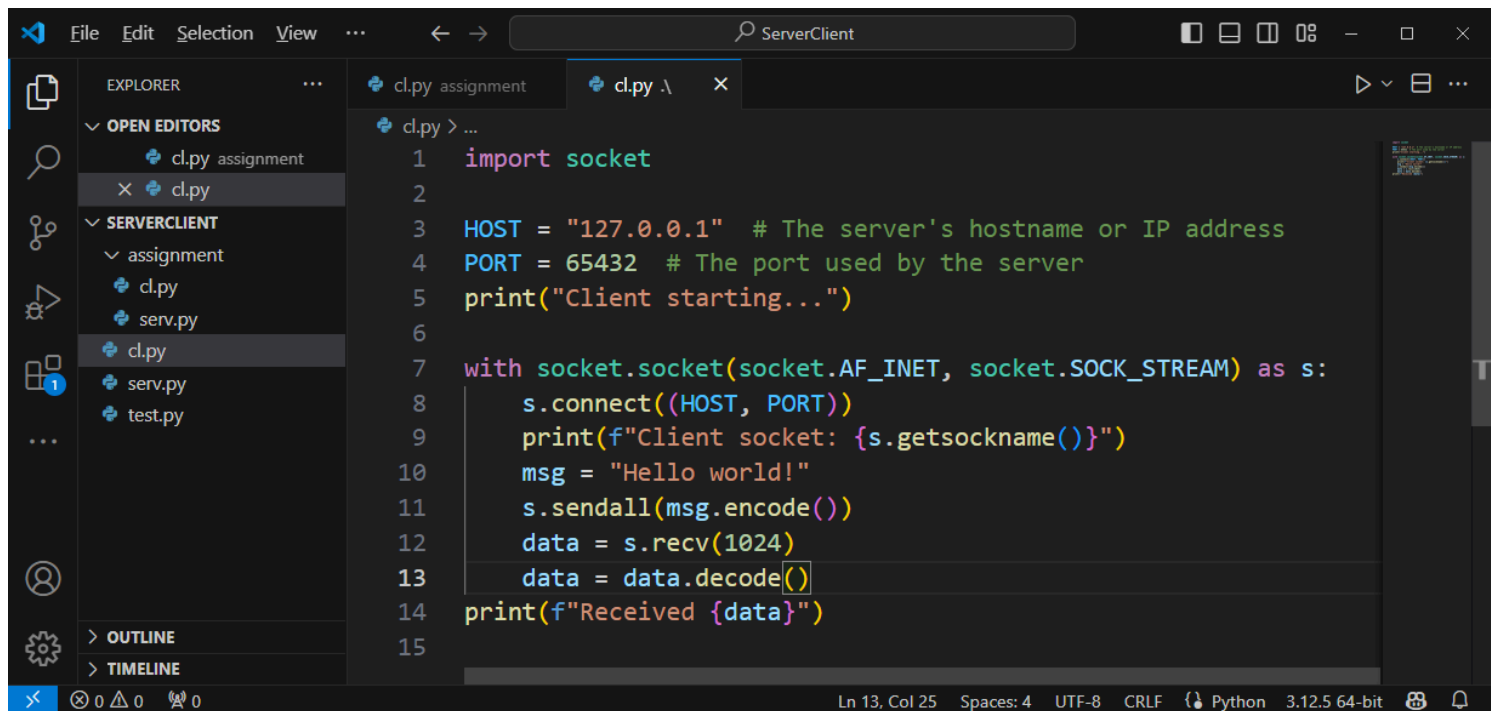
Received Digit Sum result: 4

Server returned a single digit result, Exiting Client Application

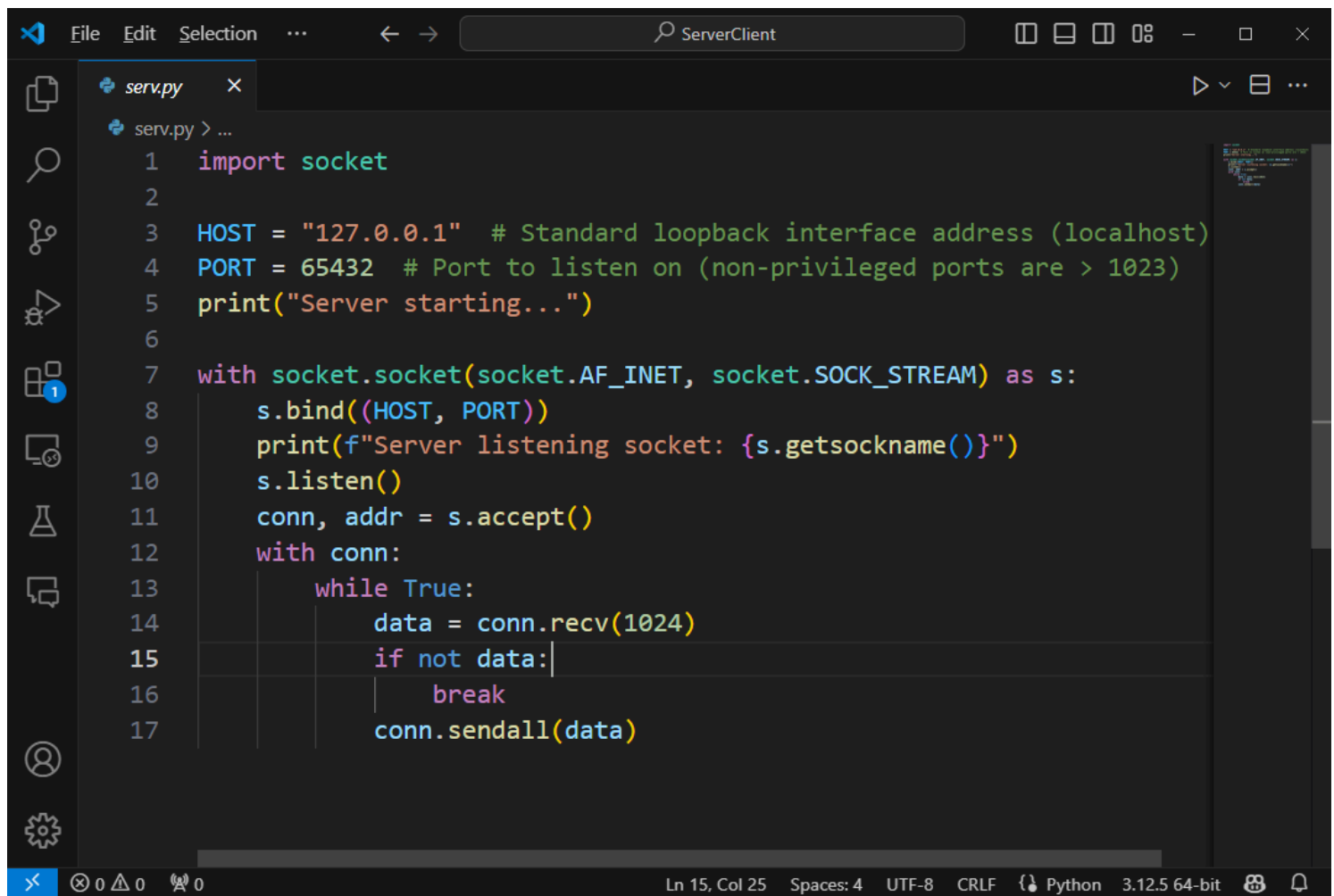
Coding Notes

- You must choose a port number greater than 1023 (to be safe, choose a server port number larger than 5000).
- Ports ID's, when bound to sockets, are system-wide values. This means that for each port ID, one and only one process can bind it at a time.
- Make sure you close every socket that you use in your program. If you abort your program, the socket may still hang around and the next time you try to bind a new socket to the port ID you previously used (but never closed), you may get an error.
- The IP 127.0.0.1 is the "Loopback" address. It always refers to the machine the process is running on. Using this address bypasses the need to remember or find your computer's actual IP address, but you can also use whatever IP address is assigned to your computer.

Basic example of client server connection.



```
1 import socket
2
3 HOST = "127.0.0.1" # The server's hostname or IP address
4 PORT = 65432 # The port used by the server
5 print("Client starting...")
6
7 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
8     s.connect((HOST, PORT))
9     print(f"Client socket: {s.getsockname()}")
10    msg = "Hello world!"
11    s.sendall(msg.encode())
12    data = s.recv(1024)
13    data = data.decode()
14    print(f"Received {data}")
15
```



```
1 import socket
2
3 HOST = "127.0.0.1" # Standard loopback interface address (localhost)
4 PORT = 65432 # Port to listen on (non-privileged ports are > 1023)
5 print("Server starting...")
6
7 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
8     s.bind((HOST, PORT))
9     print(f"Server listening socket: {s.getsockname()}")
10    s.listen()
11    conn, addr = s.accept()
12    with conn:
13        while True:
14            data = conn.recv(1024)
15            if not data:
16                break
17            conn.sendall(data)
```

Ln 15, Col 25 Spaces: 4 UTF-8 CRLF Python 3.12.5 64-bit